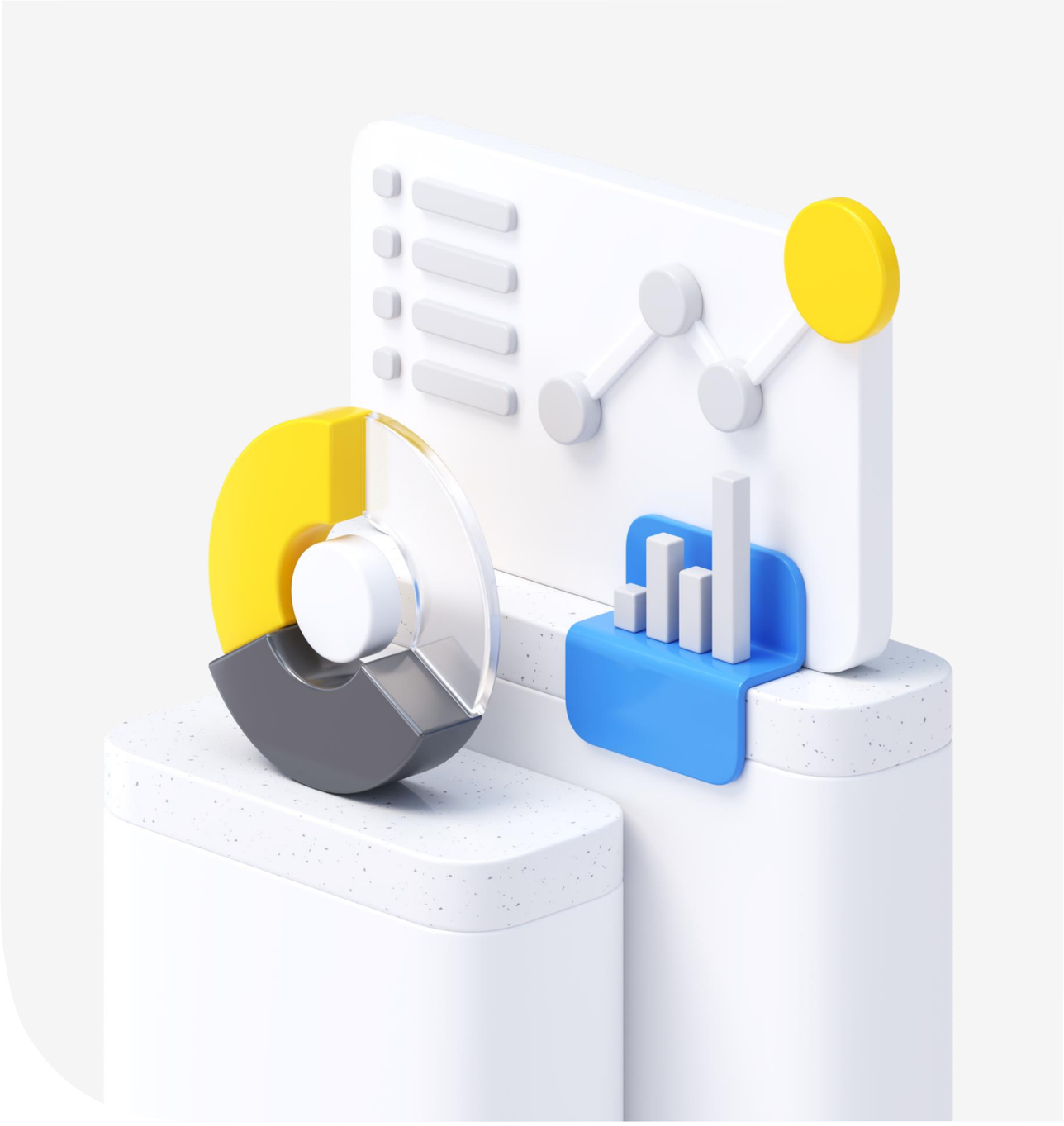
Лекция 9

Менеджеры зависимостей, GCD



План

Менеджеры зависимостей

- Зачем?
- Что использовать?
- Пример

Многопоточность

- Потоки и очереди
- Проблемы
- Решения

Менеджеры зависимостей

Что это такое?

- Инструменты для управления сторонними библиотеками
- Автоматизация установки, обновления и версионирования библиотек.

Почему это важно?

- Упрощение процесса добавления новых функций через сторонние решения.
- Поддержка актуальности библиотек.

Менеджеры зависимостей

Carthage

Менеджер зависимостей, ориентированный на создание бинарников.

Плюсы:

- Простота интеграции и легковесность.

Минусы:

- Меньше библиотек по сравнению с CocoaPods.
- Требует больше усилий для настройки.

Менеджеры зависимостей

Swift Package Manager (SPM)

Встроенный менеджер зависимостей в Xcode.

Плюсы:

- Интеграция в сам Xcode.
- Поддержка версионирования через Semantic Versioning.

Минусы:

- Ограниченное количество доступных библиотек по сравнению с CocoaPods.

Менеджеры зависимостей

CocoaPods

Наиболее популярный менеджер зависимостей для iOS и MacOS.

Позволяет легко интегрировать сторонние библиотеки через файл Podfile.

Плюсы:

- Огромное количество доступных библиотек.
- Широкая поддержка сообществом.

Минусы:

- Долгое время установки при большом количестве зависимостей.

Менеджеры зависимостей

CocoaPods

Инициализация проекта:

- pod init

Редактирование Podfile:

- pod ‘<имя_пода>’, ’версия'

Установка зависимостей:

- pod install

Открыть проект через созданный .xcworkspace вместо .xcodeproj

Profit!

Менеджеры зависимостей

Версионирование

Управление версиями библиотек, чтобы гарантировать совместимость и стабильность.

Semantic Versioning (SemVer), MAJOR.MINOR.PATCH:

- MAJOR: изменения, не совместимые с предыдущими версиями.
- MINOR: добавление новых возможностей в обратной совместимости.
- PATCH: исправления ошибок, не затрагивающие API.

Менеджеры зависимостей

Ограничения версий

pod ‘<имя_пода>’, ‘**<ВЕРСИЯ>’:**

- Не указана – используется последняя доступная версия.
- ’0.1’ – точно 0.1 версия зависимости
- '> 0.1’ – любая версия выше 0.1
- '>= 0.1’ – версия 0.1 или любая версия выше
- '< 0.1’ – любая версия ниже 0.1
- '<= 0.1’ – версия 0.1 или ниже

Оптимистичный оператор:

- '~> 0.1.2’ – версия 0.1.2 и версии до 0.2, не включая 0.2 и выше
- '~> 0.1’ – версия 0.1 и версии до 1.0, не включая 1.0 и выше
- '~> 0’ – версия 0 и версии до 1.0, не включая 1.0 и выше

Многопоточность

Много-поточность

Главный поток:

- Основной поток, на котором создаётся и исполняется пользовательский интерфейс приложения.
- Все обновления пользовательского интерфейса должны выполняться на главном потоке.

Background очереди:

- Потоки, которые работают в фоновом режиме и позволяют выполнять длительные задачи без блокировки главного потока.
- Важно использовать background очереди для задач, которые не требуют взаимодействия с пользовательским интерфейсом.

Много-поточность

Очередь vs. Поток

Очередь:

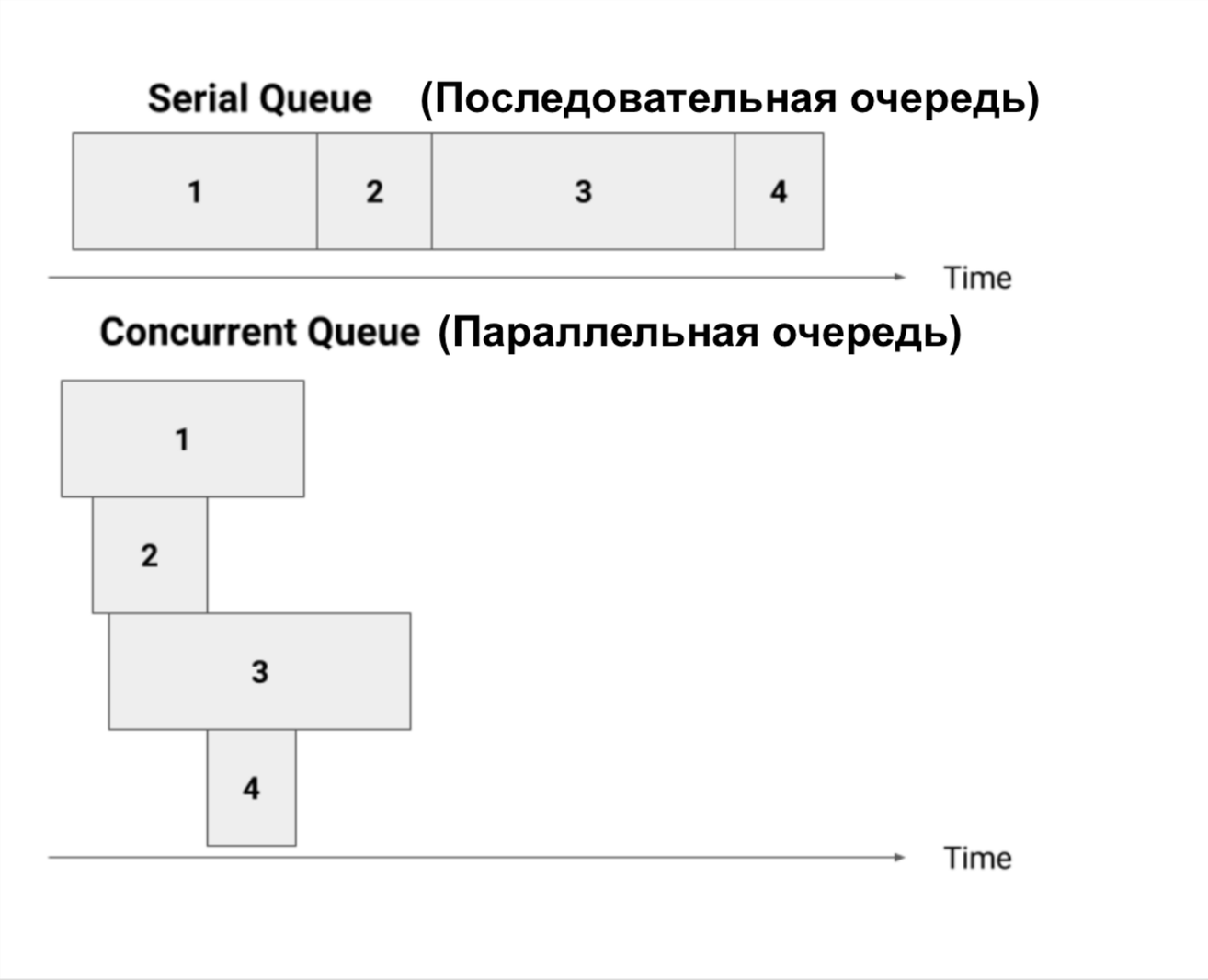
- Абстракция для управления выполнением задач.
- Задачи в очереди могут выполняться в разных потоках, упрощая управление многопоточностью. Background очередь:

Поток:

- Объект операционной системы, который выполняет код.
- Разные потоки могут выполнять код параллельно, но управление ими сложнее.

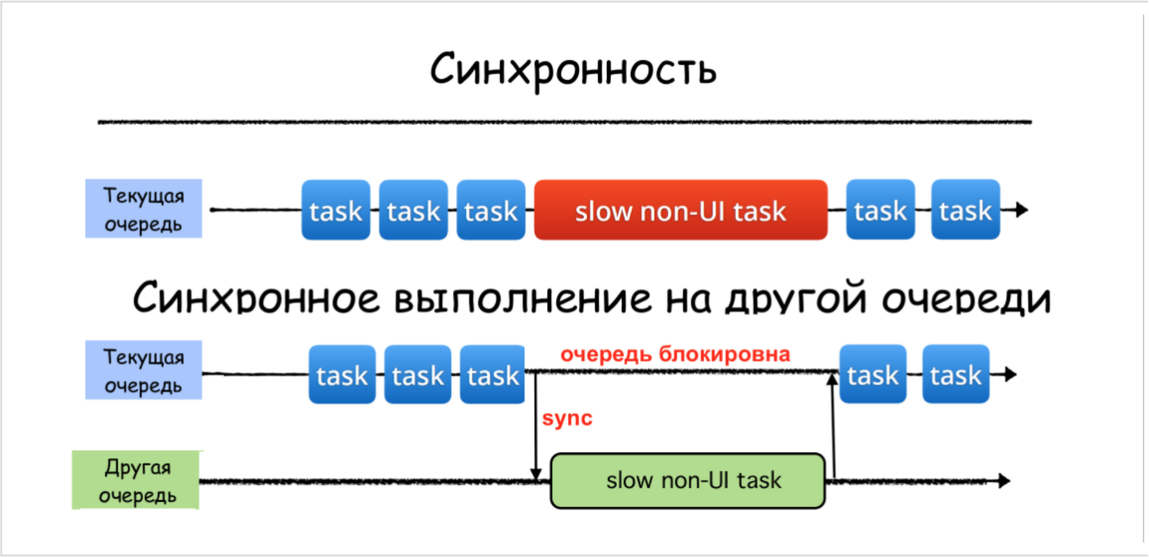
Много-поточность

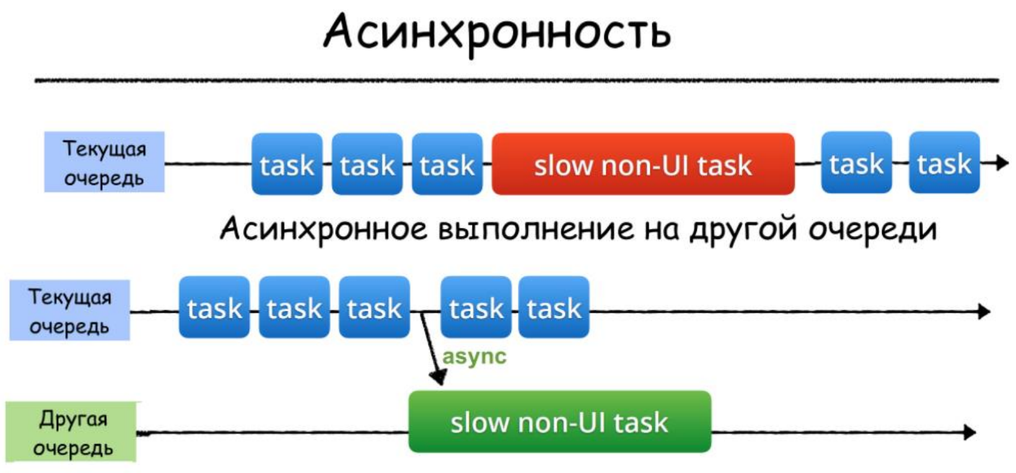
Последовательная и параллельная очередь.



Много-поточность

Sync vs Async





Много-поточность

GCD

Последовательная (serial) очередь.

```
let serialQueue = DispatchQueue(label: "mySerialQueue")

serialQueue.async {

    // Task 1

}

serialQueue.async {

    // Task 2 - выполнится только после Task 1

}
```

Много-поточность

GCD

Параллельная (concurrent) очередь.

```
let concurrentQueue = DispatchQueue(
                label: "myConcurrentQueue",
                attributes: .concurrent
)
concurrentQueue.async {
  // Task 1 - может выполняться одновременно с Task 2
}
concurrentQueue.async {
  // Task 2
}
```

Много-поточность

GCD

Глобальные очереди

```
// Главная очередь

let main = DispatchQueue.main

// наивысший приоритет

let userInteractiveQueue = DispatchQueue.global(qos: .userInteractive)

let userInitiatedQueue = DispatchQueue.global(qos: .userInitiated)

let utilityQueue = DispatchQueue.global(qos: .utility)

// самый низкий приоритет

let backgroundQueue = DispatchQueue.global(qos: .background)

// по умолчанию

let defaultQueue = DispatchQueue.global()
```

Много-поточность

Проблемы

Гонка данных (data race)

Состояние, когда один поток обращается к изменяемому объекту, в то время как другой поток записывает в него.

```
var counter = 0

DispatchQueue.concurrentPerform(iterations: 100) { _ in

    counter += 1

}

print(counter) // ???
```

Много-поточность

Проблемы

Блокировки

- os_unfair_lock – Самая быстрая блокировка. Не стоит использовать напрямую. Абстракции: UnfairLock (<= iOS 15), OSAllocatedUnfairLock (>= iOS 16)
- NSLock – Медленнее чем os_unfair_lock. Можно использовать без дополнительных абстракций. Больше возможностей.
- DispatchSemaphore - противоположностью блокировки: вместо того, чтобы заявлять права на кусок кода и препятствовать доступу других потоков к нему блокирует текущий поток пока ожидает сигнала от других.

Много-поточность

Проблемы

Взаимная блокировка (Deadlock)

Состояние, когда потоки блокируют друг друга.

```
let queue = DispatchQueue(label: "label")

queue.sync {

    queue.sync {

        // deadlock

    }

}
```

Много-поточность

Проблемы

Гонка состояний (Race condition)

Состояние, возникающее, когда время или порядок событий влияют на корректность результата.

См. Пример

Много-поточность

Группы

DispatchGroup

Инструмент для работы с несколькими задачами.

- enter() – Вход в группу, увеличение счетчика
- leave() – Выход из группы, уменьшение счетчика.
- notify(queue: DispatchQueue) – Оповещение об окончании работы группы. Сработает когда внутренний счетчик равен 0.

Много-поточность

Заключение

Выводы

- Держите UI на главном потоке.
- Избегайте долгих блокировок.
- Используйте инструменты системы под требования задачи.
- Старайтесь минимизировать доступ к данным.

Вопросы

Литература

- Podfile
- Многопоточность (concurrency) в Swift 3. GCD и Dispatch Queues
- GCD (Apple)
- Проблемы многопоточности
- Блокировки
- Alamofire

Т БАНК

Спасибо!